

Sztuczna inteligencja i systemy ekspertowe
Zadanie: Dopasowanie funkcji za pomocą sieci neuronowej

1. Cel ćwiczenia

Głównym celem ćwiczenia jest zbadanie właściwości, dynamiki zbieżności oraz skuteczności procesu uczenia sieci typu MLP w rozwiązywaniu dwóch problemów analitycznych: klasyfikacji oraz autoasocjacji.

Realizacja celu obejmuje przeprowadzenie serii eksperymentów podzielonych na dwa etapy:

- **Problem klasyfikacji wieloklasowej** z wykorzystaniem zbioru danych Irysów. W tej części badana jest zdolność sieci do generalizacji wiedzy. Kluczowym aspektem jest dobór optymalnej topologii sieci oraz właściwy podział danych na zbiór treningowy i testowy. Skuteczność wyuczonego modelu oceniana jest na podstawie wyznaczonych wskaźników jakości: macierzy pomyłek, precyzji, czułości oraz miary F_1 .
- **Problem autoasocjacji** realizowany za pomocą architektury autoenkodera o strukturze 4-2-4. Celem tego etapu jest analiza procesu redukcji rozmiaru i tworzenia ukrytych reprezentacji danych. Badania skupiają się na określeniu wpływu wejścia obciążającego (bias) na samą możliwość osiągnięcia zbieżności algorytmu. Ponadto analizowana jest szybkość uczenia sieci w zależności od kombinacji kluczowych parametrów uczenia: współczynnika nauki oraz momentum, wraz z identyfikacją warunków prowadzących do rozbieżności procesu nauki.

2. Matematyczny model działania sieci MLP

Sieć neuronowa typu Multi-layer Perceptron składa się z warstwy wejściowej, warstw ukrytych oraz warstwy wyjściowej. Przetwarzanie informacji w takiej strukturze można podzielić na dwa główne etapy: propagację sygnału w przód oraz propagację błędu wstecz. Propagacja wstecz służy do aktualizacji wag biasów przy poszczególnych neuronach.

Propagacja w przód:

Każdy neuron przetwarzający (ukryty oraz wyjściowy) w pierwszej kolejności oblicza sumę ważoną swoich wejść (tzw. pobudzenie). Dla pojedynczego neuronu w warstwie, do którego dociera n sygnałów wejściowych $x_1, x_2, \dots, x_n$ poprzez połączenia o wagach $w_1, w_2, \dots, w_n$, suma ta wyraża się wzorem:

$$s = \sum_{i=1}^n w_i x_i + b$$

gdzie b oznacza wartość obciążenia (bias), działającą jako dodatkowy sygnał o stałej wartości wejściowej równej 1 i modyfikowalnej wadze, przesuwający funkcję aktywacji.

Następnie wyznaczony potencjał s podawany jest jako argument do nieliniowej funkcji aktywacji. Zgodnie z założeniami, wykorzystano funkcję sigmoidalną o współczynniku nachylenia równym 1. Wynik y neuronu zamyka się w przedziale $(0, 1)$ i wyraża się zatem wzorem:

$$y = f(s) = \frac{1}{1 + e^{-s}}$$

Algorytm wstecznej propagacji błędów:

Proces uczenia sieci z nauczycielem polega na iteracyjnej minimalizacji błędu popełnianego przez sieć. Dla każdego zaprezentowanego wzorca, wyznaczany jest błąd globalny, w postaci błędu średniokwadratowego (MSE). Różnica między pożądaną wartością wyjściową z_i , a rzeczywistą odpowiedzią sieci y_i określa błąd lokalny:

$$d_i = z_i - y_i$$

Algorytm wstecznej propagacji błędów wykorzystuje metodę najszybszego spadku do modyfikacji wag. Kluczowym elementem jest wyznaczenie tzw. sygnału błędów δ dla każdego neuronu.

Wykorzystując własność pochodnej funkcji aktywacji, gdzie $f'(s) = y(1 - y)$, sygnał błędów dla warstwy wyjściowej wyznacza się następująco:

$$\delta_j^{(wyj)} = (z_j - y_j) \cdot y_j(1 - y_j)$$

Dla neuronów w warstwach ukrytych, które nie mają bezpośrednio zdefiniowanego pożądanego wyjścia, błąd jest propagowany wstecz na podstawie błędów neuronów z warstwy następnej, z którymi dany neuron jest połączony:

$$\delta_j^{(ukr)} = y_j(1 - y_j) \sum_k \delta_k^{(nast)} w_{kj}$$

Modyfikacja wag z momentum:

Po wyznaczeniu wartości δ dla wszystkich przetwarzających neuronów w sieci, następuje właściwa modyfikacja wag połączeń. Nowa wartość wagi obliczana jest według:

$$\Delta w_{ij}(t) = \alpha \cdot \delta_j \cdot x_i + \mu \cdot \Delta w_{ij}(t - 1)$$

$$w_{ij}(t + 1) = w_{ij}(t) + \Delta w_{ij}(t)$$

gdzie:

- $\Delta w_{ij}(t)$ – zmiana wagi połączenia między i-tym neuronem warstwy poprzedniej a j-tym neuronem warstwy obecnej w chwili t
- α – współczynnik nauki określający wielkość kroku w kierunku przeciwnym do wektora błędu
- x_i – sygnał wyjściowy i-tego neuronu z warstwy poprzedzającej
- μ – współczynnik momentum
- $\Delta w_{ij}(t - 1)$ – zmiana wagi w poprzedniej iteracji

3. Klasyfikacja zbioru Irysów

Eksperyment przeprowadzono z wykorzystaniem klasycznego zbioru danych Irysów, składającego się ze 150 próbek. Każdy wzorzec opisany jest za pomocą czterech atrybutów (długość i szerokość działki kielicha oraz płatek), a celem sieci jest przyporządkowanie analizowanego obiektu do jednej z trzech klas: Setosa, Versicolor lub Virginica. W celu weryfikacji własności generalizacyjnych sieci, zbiór podzielono losowo: 70% danych (105 próbek) stanowiło zbiór uczący, a pozostałe 30% (45 próbek) wykorzystano jako zbiór testowy do weryfikacji skuteczności modelu po zakończeniu nauki.

Przeprowadzono proces uczenia dla poniższej konfiguracji:

- Topologia: 4 neurony wejściowe - 5 neuronów ukrytych - 3 neurony wyjściowe
- Współczynnik uczenia: 0,4
- Momentum: 0,2
- Maksymalna liczba epok: 2000
- Docelowy błąd: 0,001
- Czy używano biasu: Tak
- Czy losowa kolejność danych podczas nauki: Tak

Macierz pomyłek:

Faktyczna \ Przewidywana	Setosa	Versicolor	Virginica
Setosa	14	0	0
Versicolor	0	15	0
Virginica	0	0	16

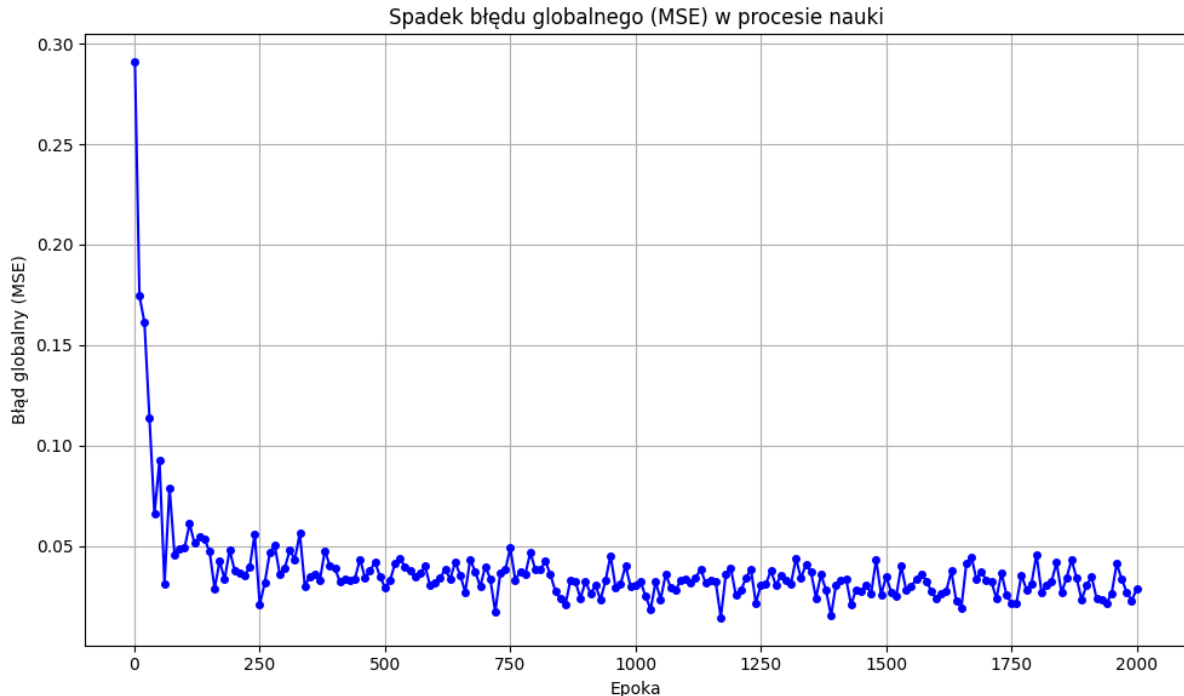
Wskaźniki jakości klasyfikacji:

Klasa	Prawdziwie Pozytywne	Precyzja	Czułość	F ₁
Setosa	14	1,00	1,00	1,00
Versicolor	15	1,00	1,00	1,00
Virginica	16	1,00	1,00	1,00

Proces uczenia, zakończony po 2000 epokach, doprowadził do osiągnięcia błędu MSE na poziomie 0,028. Uzyskany model poprawnie sklasyfikował wszystkie próbki w zbiorze testowym, osiągając 100% skuteczności. Macierz pomyłek dla zbioru testowego zawiera wartości niezerowe wyłącznie na głównej przekątnej. Brak jakichkolwiek fałszywie pozytywnych (FP) oraz fałszywie negatywnych (FN) wskazań bezpośrednio przekłada się na

maksymalne możliwe wartości precyzji, czułości oraz miary F1 dla każdego z trzech gatunków. Oznacza to, że przy tym konkretnym, losowym podziale danych, wyuczone wagi sieci wykreowały optymalne nieliniowe granice decyzyjne.

Wykres 1. Wykres nauki (epoki i ich błędy MSE) dla zbioru irysów.



4. Autoasocjacja

Charakterystyka problemu i struktura sieci:

Drugim zrealizowanym eksperymentem było badanie procesu uczenia sieci typu autoenkoder. Celem autoenkodera jest bezbłędne odtworzenie na warstwie wyjściowej wzorca podanego na warstwę wejściową. Zgodnie z założeniami, narzucono topologię sieci 4-2-4 (4 neurony wejściowe, 2 neurony ukryte, 4 neurony wyjściowe) oraz zbiór uczący składający się z 4 wektorów: (1,0,0,0), (0,1,0,0), (0,0,1,0) oraz (0,0,0,1). Kluczową trudnością tego problemu jest konieczność kompresji danych. Sieć musi zrzutować 4-wymiarową przestrzeń wejściową na 2-wymiarową przestrzeń warstwy ukrytej.

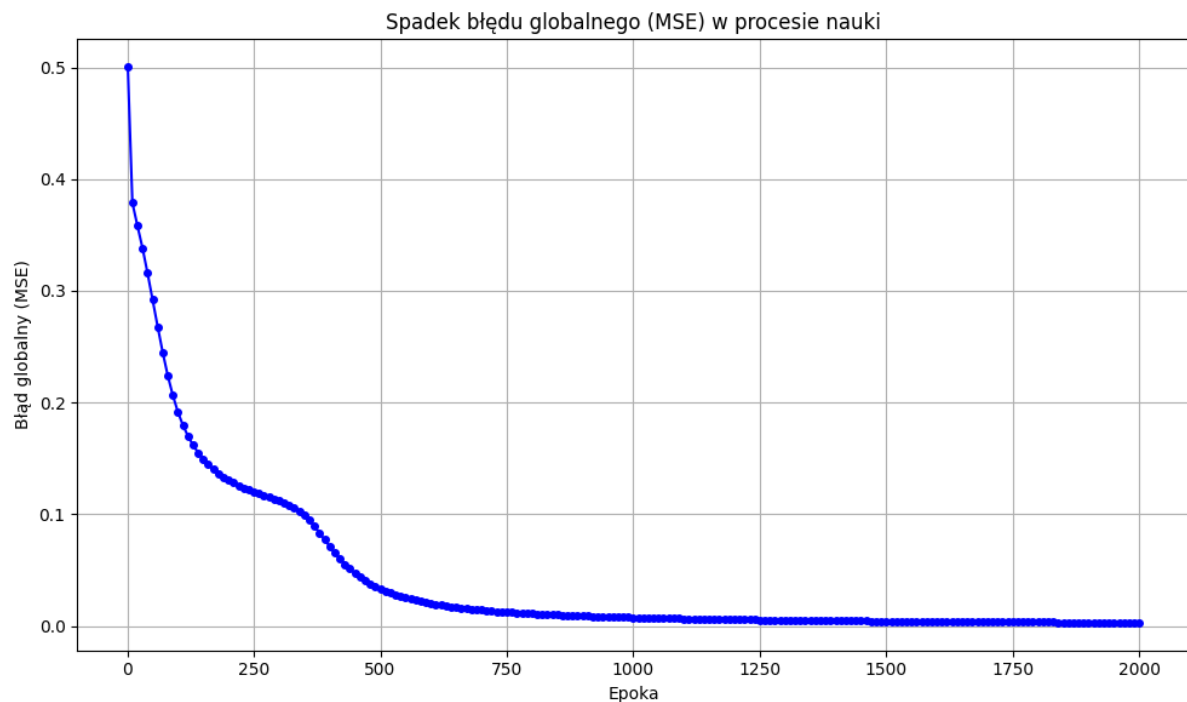
Wpływ wejścia obciążającego (bias) na proces nauki:

Nauka z biasem:

Przeprowadzono proces uczenia dla poniższej konfiguracji:

- Topologia: 4 neurony wejściowe - 2 neuronów ukrytych - 4 neurony wyjściowe
- Współczynnik uczenia: 0,6
- Momentum: 0
- Maksymalna liczba epok: 2000
- Docelowy błąd: 0,001
- Czy używano biasu: Tak
- Czy losowa kolejność danych podczas nauki: Tak

Wykres 2. Wykres nauki (epoki i ich błędy MSE) dla zbioru autoenkoder z biasem.



Analiza stanów neuronów ukrytych:

Istota działania autoenkodera uwidacznia się w wartościach sygnałów generowanych przez dwa neurony ukryte. Zarejestrowane odpowiedzi dla poszczególnych wejść prezentują się następująco:

- Wejście: [1, 0, 0, 0] daje: Stan ukryty: [0.0154, 0.982]
- Wejście: [0, 1, 0, 0] daje: Stan ukryty: [0.0137, 0.0212]
- Wejście: [0, 0, 1, 0] daje: Stan ukryty: [0.9813, 0.9785]
- Wejście: [0, 0, 0, 1] daje: Stan ukryty: [0.9659, 0.014]

Wartości te dowodzą, że sieć samoczynnie wypracowała wewnętrzny, zwarty system kodowania, przypominający 2-bitowy kod binarny. Neurony w warstwie ukrytej, dzięki nieliniowej funkcji sigmoidalnej oraz możliwości elastycznego przesuwania jej wykresu za pomocą biasu, wygenerowały 4 stany, które można w przybliżeniu zapisać jako logiczne: (0, 1), (0, 0), (1, 1) oraz (1, 0).

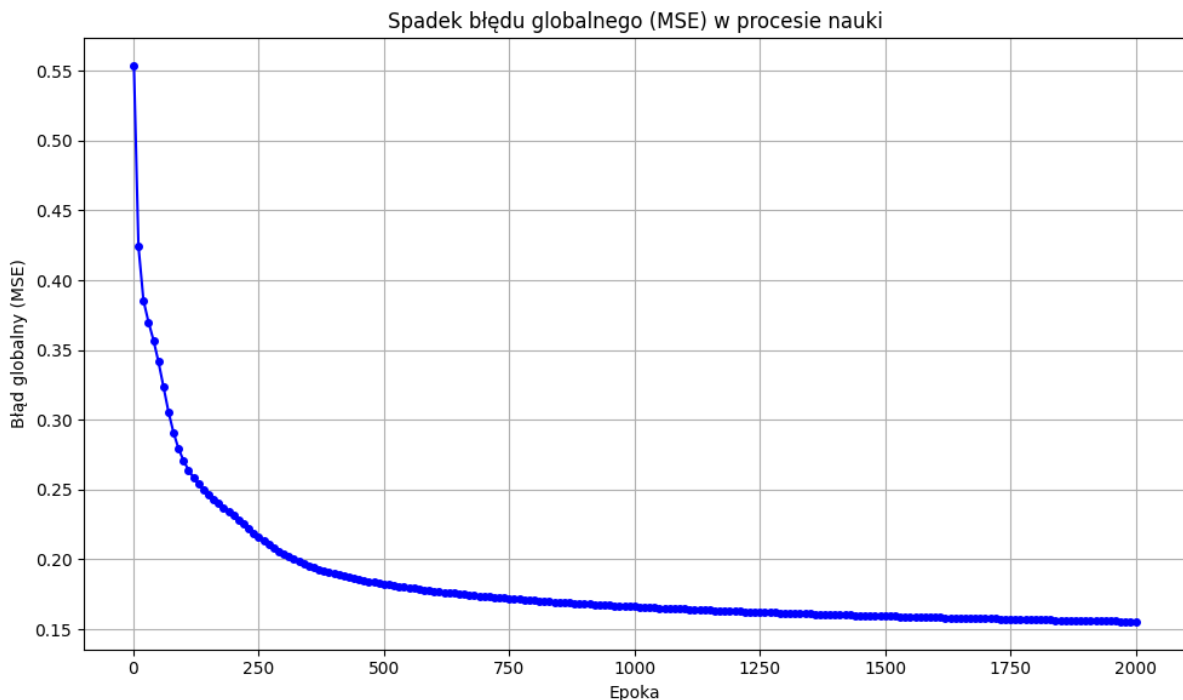
Nauka bez biasu:

Przeprowadzono proces uczenia dla poniższej konfiguracji:

- Topologia: 4 neurony wejściowe - 2 neuronów ukrytych - 4 neurony wyjściowe
- Współczynnik uczenia: 0,6
- Momentum: 0

- Maksymalna liczba epok: 2000
- Docelowy błąd: 0,001
- Czy używano biasu: Nie
- Czy losowa kolejność danych podczas nauki: Tak

Wykres 3. Wykres nauki (epoki i ich błędy mse) dla zbioru autoenkoder bez biasu.



Analiza stanów neuronów ukrytych: Skutki braku wejścia obciążającego uwidaczniają się w wartościach sygnałów generowanych przez dwa neurony ukryte. Zarejestrowane odpowiedzi dla poszczególnych wejść prezentują się następująco:

- Wejście: [1, 0, 0, 0] daje: Stan ukryty: [0.036, 0.9909]
- Wejście: [0, 1, 0, 0] daje: Stan ukryty: [0.9909, 0.0172]
- Wejście: [0, 0, 1, 0] daje: Stan ukryty: [0.181, 0.1777]
- Wejście: [0, 0, 0, 1] daje: Stan ukryty: [0.181, 0.1777]

Wartości te dowodzą, że sieć bez biasu nie potrafiła wypracować wewnętrznego, unikalnego systemu kodowania dla wszystkich wektorów. Zamiast 4 niezależnych stanów, neurony wygenerowały stany brzegowe jedynie dla pierwszego i drugiego wejścia. Dla czwartego i trzeciego wzorca proces kompresji całkowicie się załamał - sieć przypisała im niemal identyczny stan. Zjawisko to potwierdza, że bez elastycznego przesuwania wykresu nieliniowej funkcji sigmoidalnej, warstwa ukryta nakłada na siebie różne reprezentacje w przestrzeni wielowymiarowej, całkowicie uniemożliwiając ich poprawną rekonstrukcję na warstwie wyjściowej.

Badanie szybkości uczenia perceptronu:

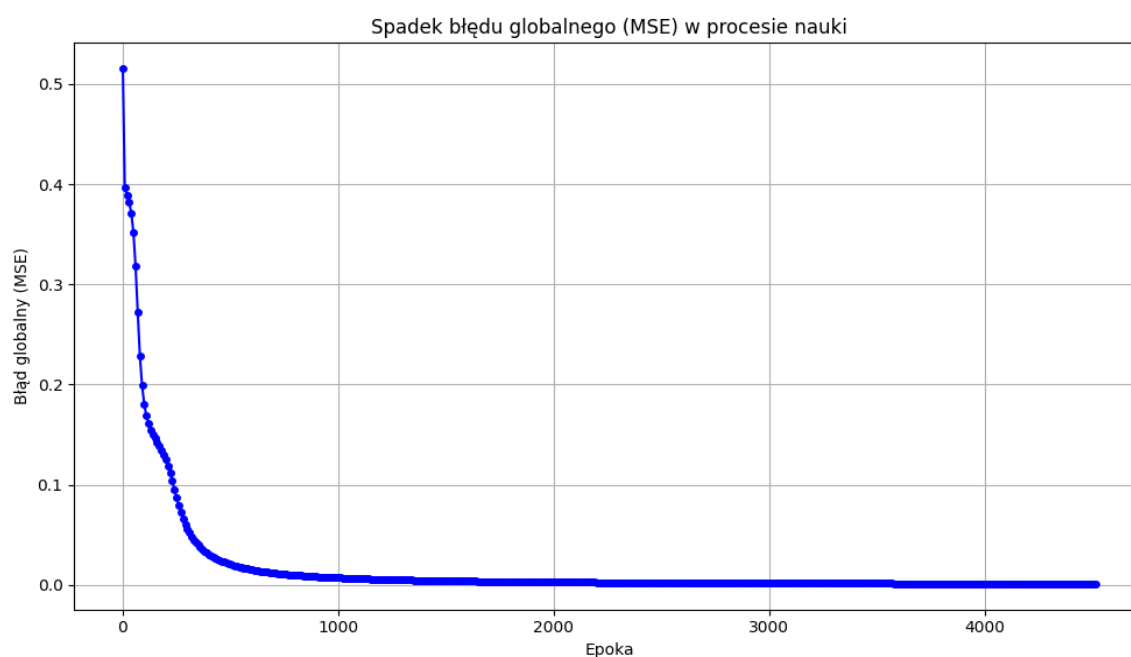
W oparciu o wariant sieci z biasem, przeprowadzono analizę wpływu doboru parametrów uczenia na szybkość minimalizacji błędu. Wyniki przedstawiające liczbę epok wymaganych do osiągnięcia narzuconego progu błędu dla poszczególnych kombinacji zestawiono w tabeli poniżej.

Przeprowadzono proces uczenia dla poniższej konfiguracji:

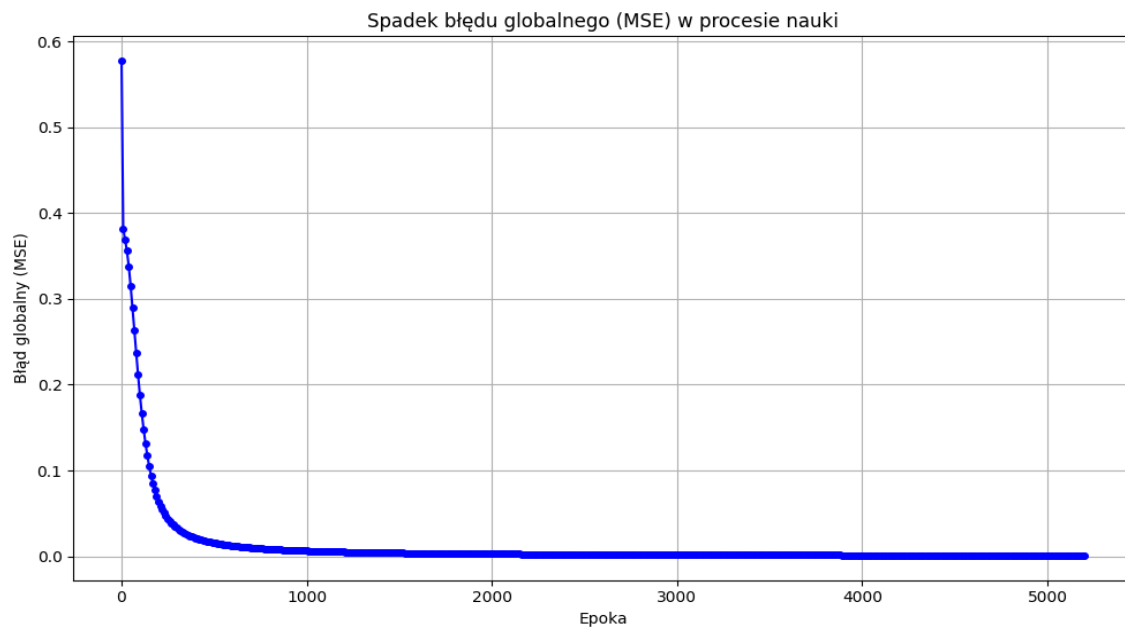
- Topologia: 4 neurony wejściowe - 2 neuronów ukrytych - 4 neurony wyjściowe
- Maksymalna liczba epok: 20000
- Docelowy błąd: 0,001
- Czy używano biasu: Tak
- Czy losowa kolejność danych podczas nauki: Tak

Współczynnik nauki (α)	Współczynnik momentum (μ)	Wynik (Liczba epok do zbieżności)
0,9	0,0	4515
0,6	0,0	5200
0,2	0,0	15383
0,9	0,6	1383
0,2	0,9	1556

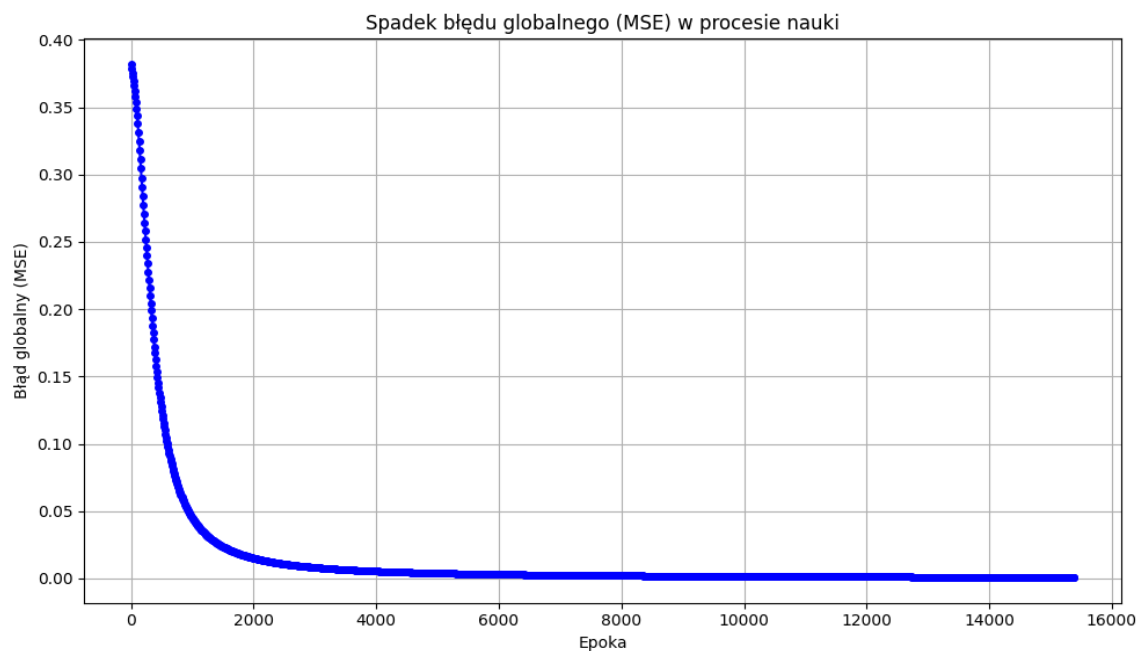
Wykres 4. Wykres nauki (epoki i ich błędy MSE) dla zbioru autoenkoder z biasem (współczynnik nauki 0.9, momentum 0.0).



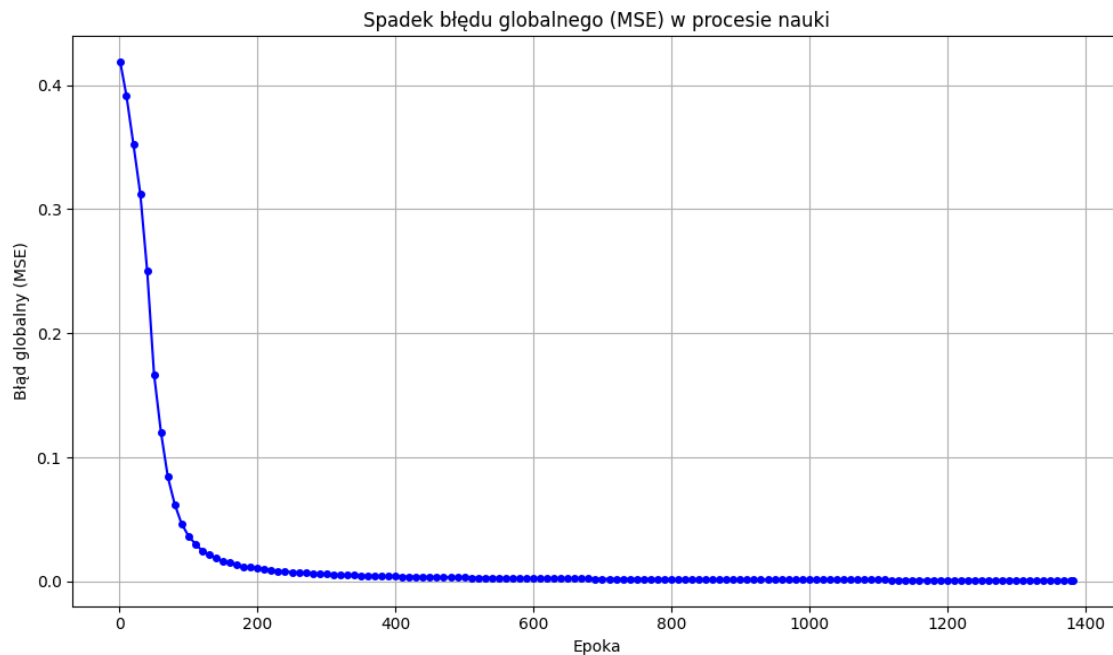
Wykres 5. Wykres nauki (epoki i ich błędy MSE) dla zbioru autoenkoder z biasem (współczynnik nauki 0.6, momentum 0.0).



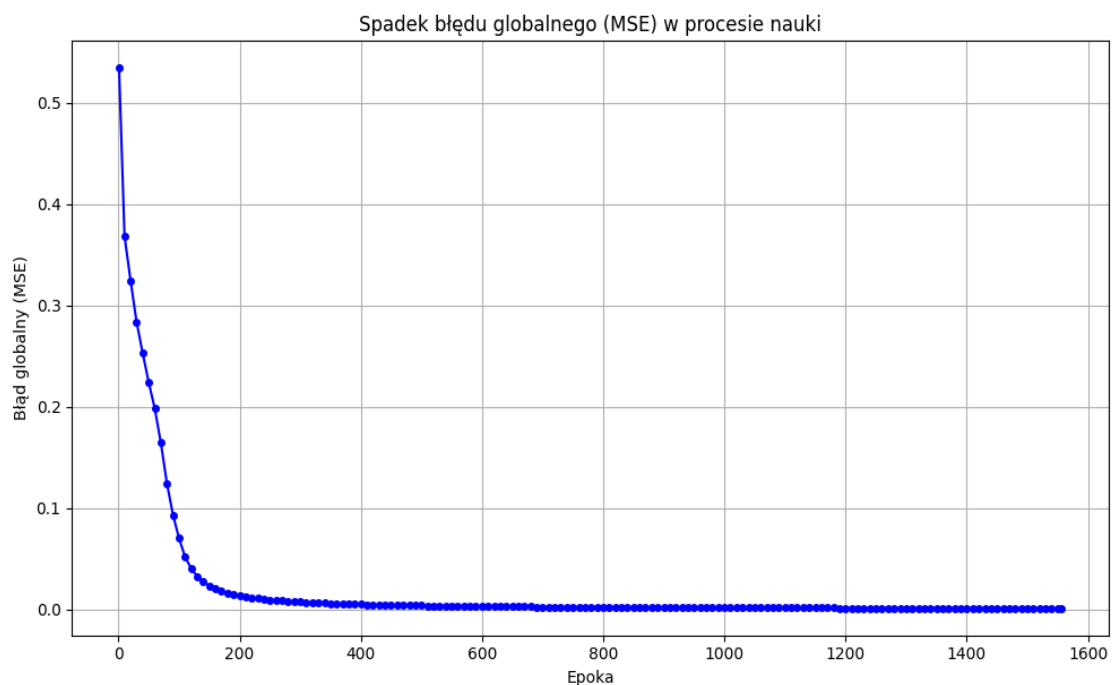
Wykres 6. Wykres nauki (epoki i ich błędy MSE) dla zbioru autoenkoder z biasem (współczynnik nauki 0.2, momentum 0.0).



Wykres 7. Wykres nauki (epoki i ich błędy MSE) dla zbioru autoenkoder z biasem (współczynnik nauki 0.9, momentum 0.6).



Wykres 8. Wykres nauki (epoki i ich błędy MSE) dla zbioru autoenkoder z biasem (współczynnik nauki 0.2, momentum 0.9).



Zestawienie wyników wyraźnie ukazuje, że poleganie na samym współczynniku nauki nie gwarantuje optymalnej zbieżności. Bardzo niska wartość współczynnika nauki bez wsparcia momentum powoduje skrajnie powolną aktualizację wag, co drastycznie

wydłuża proces nauki. Zastosowanie wysokiego współczynnika nauki znacząco przyspiesza proces jednak w ogólnym przypadku brak bezwładności niesie ryzyko utknięcia w minimach lokalnych.

Zdecydowanie najlepsze rezultaty osiągnięto dla wariantów wykorzystujących wysoki człon momentum. Kombinacja $\alpha = 0,9$ oraz $\mu = 0,6$ pozwoliła na najszybsze rozwiązanie problemu. Co więcej, wariant z bardzo niskim współczynnikiem nauki, ale silnym momentum również osiągnął znakomity czas zbieżności.

5. Wnioski

- **Zdolności klasyfikacyjne sieci:** Uzyskanie 100% poprawnych klasyfikacji na zbiorze testowym potwierdza, że sieć MLP skutecznie poradziła sobie z wyznaczeniem granic decyzyjnych w tym zadaniu. Przyjęty podział danych (70% zbiór uczący, 30% testowy) pozwolił na wypracowanie modelu, który bezbłędnie poradził sobie z rozpoznawaniem nowych próbek.
- **Wpływ biasu na reprezentację danych:** Eksperyment z autoenkoderem 4-2-4 wykazał kluczowe znaczenie wejścia obciążającego (bias) dla funkcjonowania sieci. Obecność biasu pozwala na odpowiednie przesunięcie funkcji aktywacji, co umożliwia sieci wypracowanie unikalnych, rozróżnialnych reprezentacji stanów ukrytych dla poszczególnych wzorców wejściowych.
- **Skutki braku biasu:** Brak wejścia obciążającego drastycznie obniża możliwości adaptacyjne modelu. W warunkach pozbawionych biasu sieć traci zdolność do efektywnej kompresji informacji, co objawia się przypisaniem identycznych lub zbliżonych stanów ukrytych do różnych wzorców wejściowych. W konsekwencji uniemożliwia to prawidłową rekonstrukcję danych na wyjściu sieci, co potwierdza, że bias jest niezbędnym elementem strukturalnym każdego neuronu w tej architekturze.